Ministry of Education

University of Jeddah

Collage of Computer Science and Engineering

Assignment: AI-Powered Development Platforms

CCSW 325 Software Construction

Osama Al-Ghamdi	**2340546**
Ameer Al-Ghamdi	**2340533**
Faisal Al-Orabi	**2341120**
Abdullah Al-Mazroui	**2340644**
Abdulmalik Al-Zahrani	**2340534**

1. Platform Overview

Flowise is an open-source, low-code platform for visually building applications based on Large Language Models (LLMs). It provides a drag-and-drop interface to compose chatbots, AI agents, and LLM workflows by connecting models with components such as prompts, memory, document loaders, retrievers, and external tools [1, 2]. The workflow-based design improves modularity and interpretability and supports fast iteration for both academic prototypes and real deployments [1].

The platform is self-hostable and supports executing flows through REST APIs, which enables integration into existing systems and controlled/private environments [3]. Flowise organizes its building blocks as "integrations" (nodes), allowing users to assemble end-to-end pipelines such as standard chat, Retrieval-Augmented Generation (RAG), and agentic tool-using systems [1, 4].

2. Core Concepts

In Flowise, nodes are referred to as integrations. Each integration performs a specific role (e.g., LLM, prompt template, memory, document loader, vector store, retriever, tool), and the final application is built by connecting these nodes into a directed workflow [4].

Chatflow vs Agentflow

Chatflow is commonly used for chatbots and general LLM workflows, while Agentflow (especially Agentflow V2) targets more agentic and multi-step systems, including multi-agent architectures and stateful execution [1].

Common Workflow Patterns

Common Flowise patterns include simple chat (prompt → LLM), RAG (documents → retriever → LLM), and agentic flows (LLM + tools + multi-step reasoning) [1].

Memory

Memory in Flowise enables the assistant to keep conversational context across multiple turns, improving coherence and reducing repeated questions. A memory component stores recent or summarized chat history and injects the most relevant context back into the prompt during execution [1]. This is useful for tracking user preferences and maintaining task state in multi-step conversations, but it should be configured with limits (e.g., window size or summarization) to control token cost and reduce irrelevant context [1, 4].

3. Setup Guide

Step 1: Account Registration

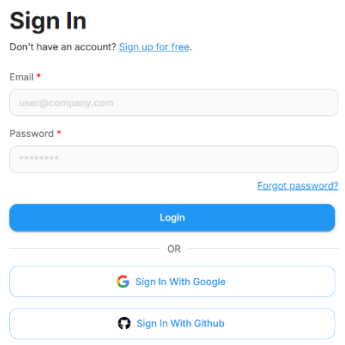

Figure 1. Flowise Cloud Registration Page

Users begin by registering for a Flowise Cloud account using an email address or Google authentication or an GitHub Account. This cloud-based setup allows immediate access to the platform without the need for local installation or environment configuration.

Step 2: Accessing the Flowise Dashboard

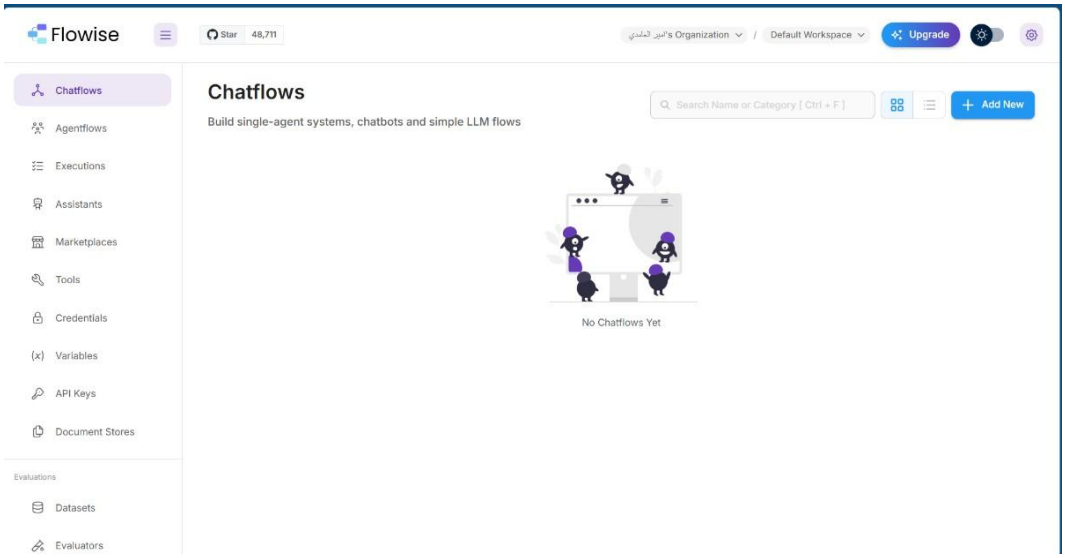

Figure 2. Flowise Home Dashboard Interface

After successful login, users are redirected to the Flowise home dashboard. From this page, users can manage their workspace, view existing projects, and navigate to the flow builder interface.

Step 3: Creating a New Agent (Flow)

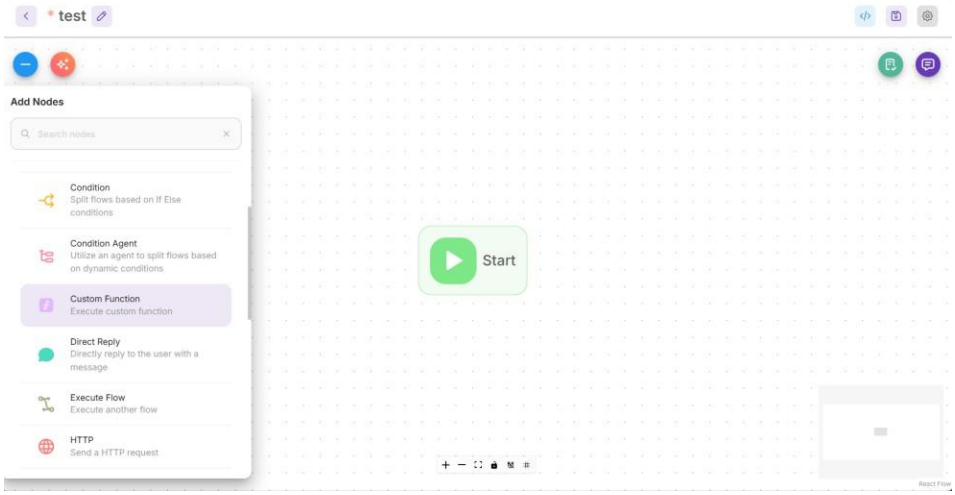

Figure 3. Visual Agent Creation Canvas in Flowise

Flowise provides a visual drag-and-drop canvas where users can create AI agents. A new flow can be initialized by starting with a blank agent, allowing full control over the workflow structure and components.

Step 4: Configuring the Language Model

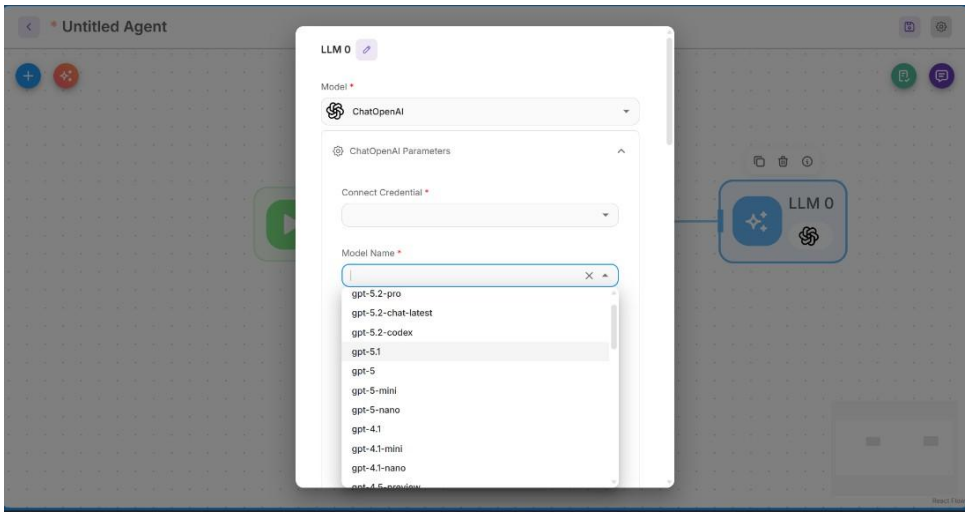

Figure 4. LLM Configuration Using ChatOpenAI

To enable AI capabilities, users configure a language model by selecting an LLM provider such as ChatOpenAI. The platform allows choosing the model version and connecting required credentials, enabling seamless integration with large language models.

Optional: Alternative Setup Methods

Flowise also supports self-hosted deployment using Docker for developers who require advanced customization or on-premises control. However, this project focuses on the cloud-based setup for simplicity and accessibility

4. Key Features

Main Capabilities

Orchestration: Visual editor supports open source and proprietary models expressions custom code and branching/looping/routing logic.

Data Ingestion & Integration: Connects to 100+ sources and tools including vector databases and memories.

Monitoring: Execution logs visual debugging and external log streaming.
Deployment: Self hosted options including air gapped deployment.
Data Processing: Data transforms filters, aggregates custom code and RAG indexing pipelines.

Pricing

Flowise pricing depends on how you use it if you self-host the open-source version it's free (Apache 2.0) If you use Flowise Cloud it's subscription based with three main tiers Free ($0/month) Starter ($35/month) and Pro ($65/month) the paid tiers mainly increase monthly prediction limits and storage and the Pro plan also includes multi workspace and user management with extra users billed at $15 per user/month.

5. Limitations

Limitations of Flowise

Scalability limitations:
Flowise is suitable for small to medium-scale applications; however, large-scale systems with high user traffic often require additional infrastructure, such as worker processes, task queues, and external databases. These requirements increase deployment complexity and operational overhead compared to lightweight, code-based solutions [1], [2].

Limited flexibility compared to code-based frameworks:
The visual, low-code nature of Flowise simplifies development but restricts fine-grained control. Highly customized logic or complex reasoning patterns may be difficult to implement using predefined nodes alone [1].

Debugging complexity in large workflows:
While Flowise provides execution logs and visual debugging tools, troubleshooting becomes more challenging as workflows grow in size and complexity, particularly with multiple branches and integrations [1].

Restricted multi-agent capabilities:
Flowise supports agent-based workflows but lacks advanced multi-agent coordination and parallel execution. Current designs rely on a single supervisor per flow, limiting scalability in agent-heavy systems [2], [9].

Dependency on the Flowise ecosystem:
Flowise is tightly integrated with specific components and frameworks (e.g., LangChain-style integrations). Extending functionality beyond supported integrations often requires custom code, reducing the advantages of a low-code approach [1], [8].

6. Comparison with Related Platforms

Comparison Between Flowise and LangChain

LangChain enables developers to construct LLM workflows entirely through code, offering high flexibility, fine-grained control, and optimization capabilities. However, it requires strong programming expertise and longer development cycles. In contrast, Flowise emphasizes ease of use and rapid development through its visual, low-code interface. As a result, Flowise is more suitable for fast prototyping and educational or business-oriented applications, while LangChain is better suited for highly customized and large-scale systems [8].

Comparison Between Flowise and LangFlow

Both Flowise and LangFlow provide visual workflow builders for LLM-based applications. LangFlow primarily focuses on simplicity and lightweight experimentation, making it suitable for quick testing and demonstrations. Flowise, on the other hand, offers a wider range of integrations, monitoring features, and deployment options, which makes it more appropriate for production-oriented and enterprise-level use cases [1].

Comparison Between Flowise and Agent-Oriented Frameworks (e.g., AutoGen and CrewAI)

Agent-oriented frameworks are designed to support autonomous agents, parallel execution, and complex collaboration patterns. Flowise supports agent-based workflows within a structured pipeline but provides more limited capabilities for advanced multi-agent coordination and parallelism. Consequently, Flowise is better suited for structured and controlled workflows, while agent-oriented frameworks are more appropriate for research-focused or highly autonomous agent systems [9], [10].

7. Use Case

Use Case: Ecommerce Customer Support AI Companion

Flowise can be utilized to implement an AI-powered customer support companion for ecommerce platforms. By leveraging FAQ documents and structured knowledge bases, the AI system can provide accurate, context-aware responses to user queries. The system is designed using Flowise's visual workflow builder, integrating document retrieval and LLM reasoning, and is deployed through an embedded chat interface or API [1, 3, 7].

8. References

[1] FlowiseAI, "Flowise Documentation," 2025. [Online]. Available: https://docs.flowiseai.com/

[2] FlowiseAI, "Flowise: Build AI Agents, Visually," GitHub Repository, 2025. [Online]. Available: https://github.com/FlowiseAI/Flowise

[3] FlowiseAI, "API Reference — Chatflows," 2025. [Online]. Available: https://docs.flowiseai.com/api-reference/chatflows

[4] FlowiseAI, "Integrations," 2025. [Online]. Available: https://docs.flowiseai.com/integrations

[5] FlowiseAI, "FlowiseAI Website (Quick Start)," 2025. [Online]. Available: https://flowiseai.com/

[6] FlowiseAI, "flowiseai/flowise," Docker Hub. [Online]. Available: https://hub.docker.com/r/flowiseai/flowise

[7] FlowiseAI, "FlowiseChatEmbed," GitHub Repository. [Online]. Available: https://github.com/FlowiseAI/FlowiseChatEmbed

[8] IBM, "What Is LangChain?," IBM Think, 2025. [Online]. Available: https://www.ibm.com/think/topics/langchain

[9] J. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.

[10] CrewAI, "CrewAI Documentation," 2024. [Online]. Available: https://docs.crewai.com/